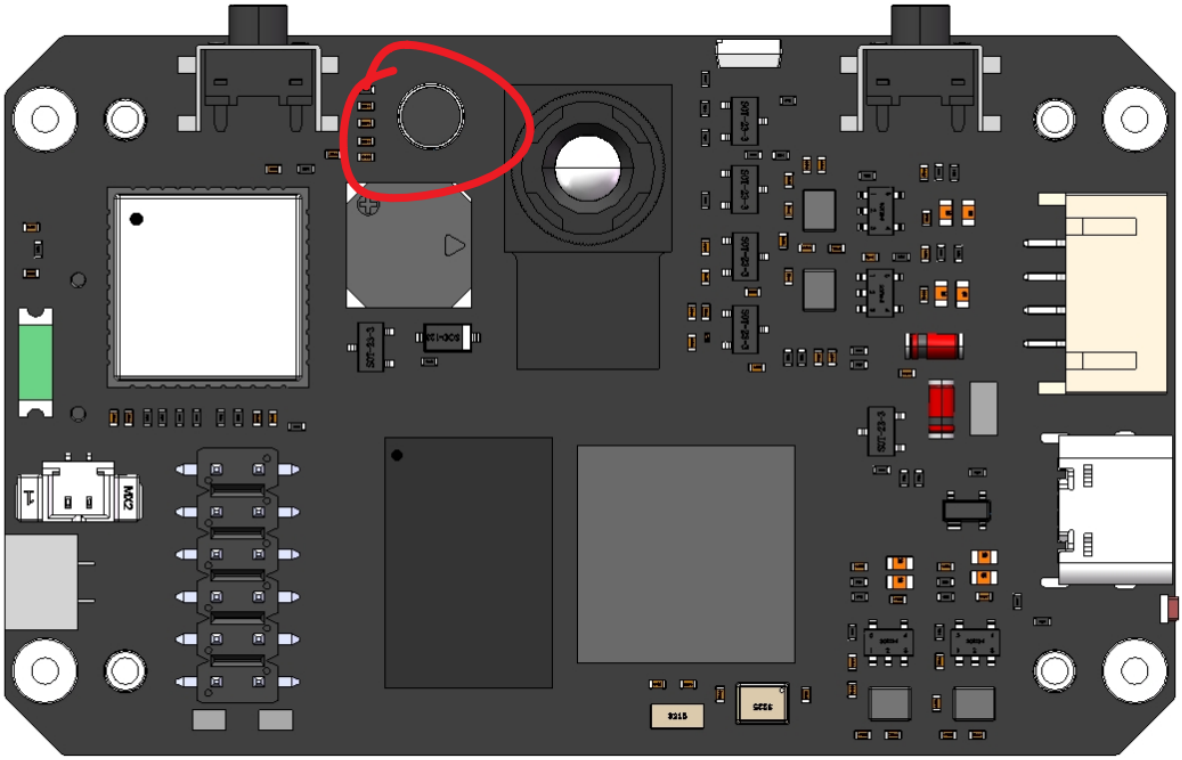
声音录制

声音录制
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构

例程实验效果简介

这一节我们用K230上面板载的麦克风进行声音录制



本节例程代码位于：【源码汇总 / 10.Media / 01.record_audio.py】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

点击CanMV IDE左下角的运行按钮，K230会开始录制10秒音频，录制结果将保存在

【/data/audio/audio.wav】(也可以自己修改)

代码讲解

> 为了方便大家快速调用音频录制功能，我们将录音功能封装成了AudioRecorder模块

完整代码

```
import os
from media.media import *
from media.pyaudio import *
import media.wave as wave

class AudioRecorder:
    """
    音频录制类
    Audio recorder class
    """
    def __init__(self):
        """
        初始化音频参数
        Initialize audio parameters
        """
#         self.CHUNK = 44100 // 25    # 每个缓冲区的帧数 / Frames per buffer
        self.FORMAT = paInt16       # 采样格式为16位整型 / 16-bit integer sampling
format
        self.CHANNELS = 1           # 单声道 / Mono channel
        self.RATE = 44100          # 采样率44.1kHz / 44.1kHz sampling rate
#         self.RATE = 16000
        self.CHUNK = self.RATE // 25    # 每个缓冲区的帧数 / Frames per buffer
        self.frames = []            # 存储录音帧 / Store recorded frames
        self.is_recording = False   # 录音状态标志 / Recording status flag

        # 初始化PyAudio / Initialize PyAudio
        self.p = PyAudio()
        self.p.initialize(self.CHUNK)
        MediaManager.init()

    def exit_check(self):
        """
        检查是否有退出信号
        Check if there is an exit signal
        """
        try:
            os.exitpoint()
        except KeyboardInterrupt as e:
            print("user stop: ", e)
            return True
        return False

    def record(self, filename, duration, left_volume=85, right_volume=85,
ans=False):
        """
        录制音频
        Record audio

        参数 / Parameters:
            filename: 音频保存路径 / Path to save audio file
            duration: 录制时长(秒) / Recording duration in seconds
            left_volume: 左声道音量 / Left channel volume
            right_volume: 右声道音量 / Right channel volume
            ans: 是否启用音频3A功能：自动噪声抑制(ANS) / Open Ans or not
        """
```

```
        try:
            # 打开音频输入流 / Open audio input stream
            self.input_stream = self.p.open(
                format=self.FORMAT,
                channels=self.CHANNELS,
                rate=self.RATE,
                input=True,
                frames_per_buffer=self.CHUNK
            )

            # 设置音量 / Set volume
            self.input_stream.volume(LEFT, left_volume)
            self.input_stream.volume(RIGHT, right_volume)

            if(ans):
                self.input_stream.enable_audio3a(AUDIO_3A_ENABLE_ANS)

            print("start record...")
            self.frames = []
            self.is_recording = True

            # 开始录制 / Start recording
            for i in range(0, int(self.RATE / self.CHUNK * duration)):
                if not self.is_recording or self.exit_check():
                    break
                data = self.input_stream.read()
                self.frames.append(data)

            print("stop record...")

            # 保存为WAV文件 / Save as WAV file
            self._save_to_wav(filename)

        except BaseException as e:
            print(f"Exception {e}")
        finally:
            self.stop()

    def stop(self):
        """
        停止录音并清理资源
        Stop recording and cleanup resources
        """
        self.is_recording = False
        if hasattr(self, 'input_stream'):
            self.input_stream.stop_stream()
            self.input_stream.close()
        self.p.terminate()
        MediaManager.deinit()

    def _save_to_wav(self, filename):
        """
        保存录音为WAV文件
        Save recording as WAV file

        参数 / Parameters:
            filename: 保存路径 / Save path
        """
```

```
        wf = wave.open(filename, 'wb')
        wf.set_channels(self.CHANNELS)
        wf.set_sampwidth(self.p.get_sample_size(self.FORMAT))
        wf.set_framerate(self.RATE)
        wf.write_frames(b''.join(self.frames))
        wf.close()


# save image raw data, use 7yuv to preview

def ensure_dir(directory):
    """
    递归创建目录，适用于MicroPython环境
    """
    # 如果目录为空字符串或根目录，直接返回
    if not directory or directory == '/':
        return

    # 处理路径分隔符，确保使用标准格式
    directory = directory.rstrip('/')

    try:
        # 尝试获取目录状态，如果目录存在就直接返回
        print(os.stat(directory))
        print(f'目录已存在 The Folder already exists: {directory}')
        return
    except OSError:
        # 目录不存在，需要创建
        # 分割路径以获取父目录
        if '/' in directory:
            parent = directory[:directory.rindex('/')]
            if parent and parent != directory:  # 避免无限递归
                ensure_dir(parent)

        try:
            os.mkdir(directory)
            print(f'已创建目录 Created Folder: {directory}')
        except OSError as e:
            # 可能是并发创建导致的冲突，再次检查目录是否存在
            try:
                os.stat(directory)
                print(f'目录已被其他进程创建 The Directory has been created by
another process: {directory}')
            except:
                # 如果仍然不存在，则确实出错了
                print(f'创建目录时出错 An error occurred while creating the
directory: {e}')
    except Exception as e:
        print(f'处理目录时出错 An error occurred while processing the directory:
{e}')

# 使用示例 / Usage example
if __name__ == "__main__":
    os.exitpoint(os.EXITPOINT_ENABLE)
    save_path = "/data/audio/"
    # 目录不存在时创建 / create if not exist
    ensure_dir(save_path)
```

```
    recorder = AudioRecorder()
    print("音频录制开始 Recording start...")
    recorder.record(save_path + 'audio.wav', 10)
    print("音频录制完成 Recording complete...")
```

代码结构

类的初始化部分：

- 设置了音频参数：44.1kHz采样率、16位采样格式、单声道
- 缓冲区大小设为每秒约25帧(44100/25)
- 初始化PyAudio和媒体管理器

主要方法：

- `exit_check()`：检测退出信号，支持用户中断录制
- `record()`：核心录制方法
  - 打开音频输入流
  - 设置左右声道音量
  - 按设定duration循环录制音频数据
  - 将录制数据保存为WAV文件
- `stop()`：停止录制并清理资源
- `_save_to_wav()`：将录制的音频数据保存为WAV格式文件

使用示例：

- 创建录音机实例
- 录制10秒音频
- 保存到指定路径

这是一个面向对象的音频录制实现，提供了完整的录制流程控制，包括：

- 录制参数配置
- 音量控制
- 异常处理
- 资源管理
- WAV格式保存